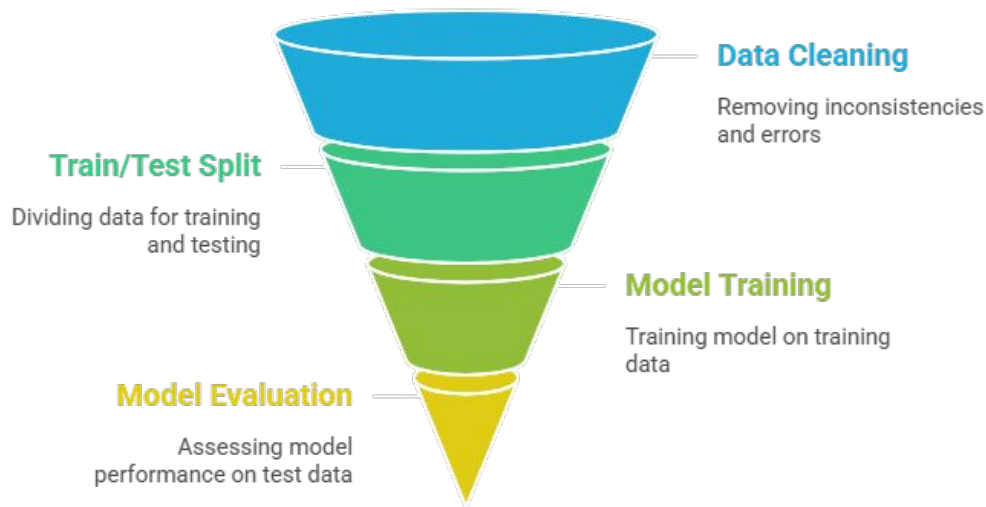

Heart Disease Risk Prediction using ML

End to End ML pipeline with FastAPI Deployment
Presented by: Haitao Wang, Ritu Arora

Agenda

1. Problem Statement & Dataset
2. Machine Learning Fundamentals
3. Data Ingestion, Transformation & Feature Engineering
4. Model Training & Hyperparameter Tuning
5. Evaluation using ROC AUC
6. Deployment using FastAPI
7. Challenges & Takeaways

Machine Learning Model Development Funnel



Problem Statement

Motivation:

- Cardiovascular diseases are the leading cause of death globally.
- Early prediction can save lives and reduce healthcare costs.

Goal:

- Build a binary classifier: Predict **Heart_Disease** (Yes/No).

Introduction to Machine Learning

Supervised Learning:

You provide the model with labeled data — it learns the mapping from input features **X** to output **Y**.

Classification Problem:

Our target variable is binary (0: No heart disease, 1: Yes).

Mathematical Viewpoint:

$$f : \mathbb{R}^n \rightarrow \{0, 1\}, \quad \text{where } f \text{ is learned from data}$$

The Dataset - What are We Working with?

~300,000 health records

Source: BRFSS Survey (Behavioral Risk Factor Surveillance System)

Features:

- Categorical: Sex, Diabetes Type, General Health
- Numerical: BMI, Alcohol Consumption, etc.

Target: Heart_Disease (Yes/No)

Project Architecture Overview

```
src/
├── components/
│   ├── data_ingestion.py
│   ├── data_transformation.py
│   ├── model_trainer.py
│   ├── model_evaluator.py
│   └── threshold_tuner.py
├── utils.py
├── exception.py
├── logger.py
├── app.py (FastAPI)
└── artifacts/ (Saved models)
```

Step 1 - Data Ingestion

Responsibilities:

- Read cleaned CSV data
- Perform train/test split (80-20)
- Save **train.csv, test.csv, raw.csv**

```
df = pd.read_csv('notebook/data/cleaned_data.csv')
```

```
train_set, test_set = train_test_split(df, test_size=0.2, random_state=42)
```

Step 2 - Data Preprocessing Pipeline

We handled 3 types of data:

1. **Numerical Columns:** Scaled using *StandardScaler*
2. **Binary Categorical:** Encoded with custom *BinaryEncoder*
3. **Ordinal Categorical:** Encoded with *OrdinalEncoder* and custom order

Used *ColumnTransformer* to combine all preprocessing.

Preprocessing - Why and How?

Why preprocess?

- ML models need numerical input
- Different scales hurt convergence
- Skewed features need transformation

How?

- Standardization:

$$z = \frac{x - \mu}{\sigma}$$

- Log Transform to fix skewness:

$$\log(x + 1)$$

Feature Engineering (Log Features)

We added 6 log-transformed features to improve model performance on skewed distributions:

```
df['BMI_log'] = np.log1p(df['BMI'])
```

Features like BMI and alcohol consumption are not normally distributed. Log transforms help certain models generalize better.

Target Variable Encoding

```
train_df['Heart_Disease'] = train_df['Heart_Disease'].map({'Yes': 1, 'No': 0})
```

Why important?

- Classification models like logistic regression need binary numerical outputs.

Step 3 - Model Training: Which Models?

Model Name	Type
Logistic Regression	Linear
Decision Tree	Non-linear
Random Forest	Ensemble
K-Nearest Neighbors	Distance-based
Gradient Boosting	Ensemble
XGBoost	Ensemble (GBM)
CatBoost	Categorical GBM
AdaBoost	Boosting

Model Math: Logistic Regression

- Predicts probabilities using the sigmoid function

$$P(y = 1 | x) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

- Loss Function: Log-loss / Binary Cross-Entropy

$$\text{Loss} = - [y \log(p) + (1 - y) \log(1 - p)]$$

y is the true label (0 or 1)

p is the predicted probability $P(Y=1 | X)$

This formula is applied to each training example and then averaged.

Model Math: Decision Trees

- Tree structure that splits data on features
- Greedy algorithm: minimizes Gini impurity / entropy at each split

$$Gini = 1 - \sum p_i^2$$

Limitation: Overfitting

Model Math: Ensemble Models (Random Forest, Boosting)

Random Forest: Bagging multiple decision trees

Gradient Boosting: Sequentially reduces error

$$F_m(x) = F_{m-1}(x) + h_m(x)$$

- $F_{m-1}(x)$: the ensemble model built up to step $m-1$
- $h_m(x)$: the new weak learner (usually a small decision tree) added at step m
- $F_m(x)$: the updated model after adding the m -th learner

Each new learner is trained to correct the **errors made by the current model**

CatBoost: Efficiently handles categorical variables internally

Hyperparameter Tuning - GridSearchCV

- We used **GridSearchCV** with 5-fold cross-validation:
gs = GridSearchCV(model, param_grid=params, scoring='roc_auc', cv=5)
- Searches best parameter combinations
- ROC AUC used as metric

Model Evaluation

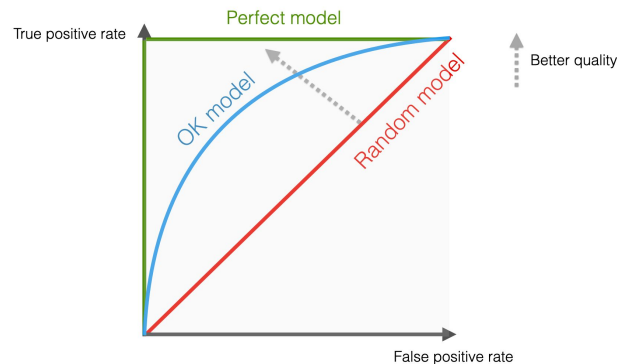
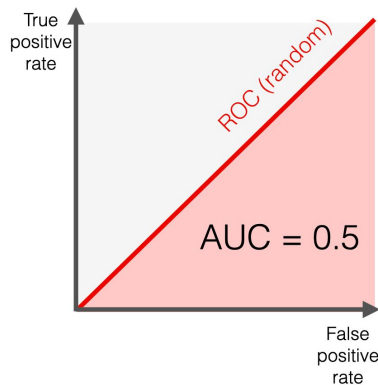
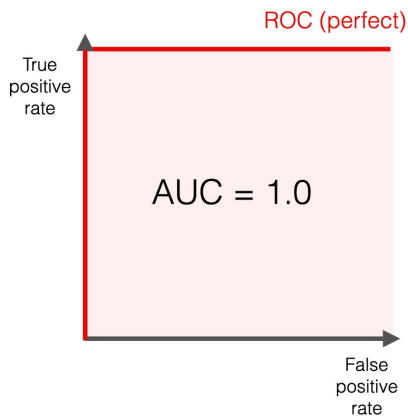
$$AUC = \int_0^1 TPR(FPR) dFPR$$

ROC Curve plots TPR vs FPR.

AUC (Area Under Curve) shows model's ability to separate classes.

- AUC = 0.5 → Random
- AUC = 1.0 → Perfect

<https://www.evidentlyai.com/classification-metrics/explain-roc-curve>



Our Results - Best Model

Model	ROC AUC
Logistic Reg.	0.79
Random Forest	0.85
XGBoost	0.87
CatBoost	0.88

Chosen Model: **CatBoostClassifier**

Step 4 - FastAPI Deployment

Created a FastAPI server

Endpoint: **/predict**

Accepts JSON input

Returns: predicted class + probability

```
@app.post('/predict')
def predict(data: PatientData):
    df = pd.DataFrame([data.dict()])[columns_order]
    transformed = preprocessor.transform(df)
    probability = model.predict_proba(transformed)[0][1]
    return {"prediction": int(probability > 0.5)}
```

Architecture & Workflow Overview

Modular Architecture: Clean Separation of Concerns

- data_ingestion.py → Data loading & splitting
- data_transformation.py → Preprocessing & feature engineering
- model_trainer.py → Training, evaluation & selection
- prediction.py (FastAPI) → Request handling & inference
- utils.py → Common helpers (save/load, evaluation, etc.)

Architecture & Workflow Overview (contd..)

Request-Response Workflow

1. Frontend / API client sends patient data as JSON
2. FastAPI receives and validates input
3. Data is preprocessed using saved pipeline (pkl)
4. Model makes prediction & returns probability
5. API sends back response (prediction + confidence)

Future Improvements & DevOps Integration

CI/CD with GitHub Actions

- Automate testing, linting, and deployment
- Trigger on push to main branch

Docker Containerization

- Package app + dependencies into a Docker image
- Ensures reproducible builds across environments

Deploy on AWS EC2

- Host FastAPI backend on a cloud VM - Use Nginx as reverse proxy

SHAP - Model Explainability

SHAP (SHapley Additive exPlanations)

What is SHAP?

- A unified framework for explaining predictions
- Based on game theory (Shapley values)

Why SHAP?

- Identifies feature importance globally & locally
- Helps build trust in model decisions
- Essential for healthcare ML models

Helps make the "black box" transparent.

Final Thoughts & Takeaway

Learned end-to-end ML pipeline

Used production tools: FastAPI, GridSearchCV

CatBoost stood out for real-world tabular data

Full-stack meets ML — practical synergy!

data_transformation.py

```
def get_preprocessor(self):
    try:
        logging.info("Preparing preprocessing pipelines for both numerical and categorical data")

        # Binary features still in object type
        binary_cols = ['Exercise', 'Skin_Cancer', 'Other_Cancer',
                       'Depression', 'Arthritis', 'Smoking_History']

        # Categorical Ordinal features
        ordinal_cols = ['General_Health', 'Checkup', 'Age_Category', 'Sex', 'Diabetes']

        # Custom order for ordinal encoder
        ordinal_mapping = [
            ['Poor', 'Fair', 'Good', 'Very Good', 'Excellent'],           # General_Health
            ['Never', '5 or more years ago', 'Within the past 5 years',
             'Within the past 2 years', 'Within the past year'],          # Checkup
            ['18-24', '25-29', '30-34', '35-39', '40-44',
             '45-49', '50-54', '55-59', '60-64', '65-69',
             '70-74', '75-79', '80+'],                                     # Age_Category
            ['Female', 'Male'],                                           # Sex
            ['No', 'No, pre-diabetes or borderline diabetes',
             'Yes, but female told only during pregnancy', 'Yes']        # Diabetes
        ]

        # Numerical columns (already include log-transformed ones)
        num_cols = [
            'Height_(cm)', 'Weight_(kg)', 'BMI', 'Alcohol_Consumption',
            'Fruit_Consumption', 'Green_Vegetables_Consumption', 'FriedPotato_Consumption',
            'Weight_(kg)_log', 'BMI_log', 'Alcohol_Consumption_log', 'Fruit_Consumption_log',
            'Green_Vegetables_Consumption_log', 'FriedPotato_Consumption_log'
        ]
```

Calculator.jsx

```
const HEALTH_OPTIONS = ["Poor", "Fair", "Good", "Very Good", "Excellent"];
const AGE_GROUPS = ["18-24", "25-29", "30-34", "35-39", "40-44", "45-49", "50-54", "55-59", "60-64", "65-69", "70-74", "75-79", "80+"];
const CHECKUP_OPTIONS = ["Never", "5 or more years ago", "Within the past 5 years", "Within the past 2 years", "Within the past year"];
const DIABETES_OPTIONS = ["No", "No, pre-diabetes or borderline diabetes", "Yes, but female told only during pregnancy", "Yes"];
const SEX_OPTIONS = ["Female", "Male"];

const schema = z.object({
  General_Health: z.enum(HEALTH_OPTIONS),
  Checkup: z.enum(CHECKUP_OPTIONS),
  Exercise: z.enum(["0", "1"]),
  Skin_Cancer: z.enum(["0", "1"]),
  Other_Cancer: z.enum(["0", "1"]),
  Depression: z.enum(["0", "1"]),
  Diabetes: z.enum(DIABETES_OPTIONS),
  Arthritis: z.enum(["0", "1"]),
  Sex: z.enum(SEX_OPTIONS),
  Age_Category: z.enum(AGE_GROUPS),

  Height_cm: z.coerce.number().min(0, { message: "Must be ≥ 0" }),
  Weight_kg: z.coerce.number().min(0, { message: "Must be ≥ 0" }),

  Smoking_History: z.enum(["0", "1"]),
  Alcohol_Consumption: z.coerce.number().min(0),
  Fruit_Consumption: z.coerce.number().min(0),
  Green_Vegetables_Consumption: z.coerce.number().min(0),
  FriedPotato_Consumption: z.coerce.number().min(0),
});
```

Calculator.jsx

```
const onSubmit = async (v) => {  
  const yn = (x) => (String(x) === "1" ? "Yes" : "No");  
  
  const payload = {  
    General_Health: v.General_Health,  
    Checkup: v.Checkup,  
  
    Exercise: yn(v.Exercise),  
    Skin_Cancer: yn(v.Skin_Cancer),  
    Other_Cancer: yn(v.Other_Cancer),  
    Depression: yn(v.Depression),  
    Arthritis: yn(v.Arthritis),  
    Smoking_History: yn(v.Smoking_History),  
  
    Diabetes: v.Diabetes,  
    Sex: v.Sex,  
    Age_Category: v.Age_Category,  
  
    Height_cm: Number(v.Height_cm),  
    Weight_kg: Number(v.Weight_kg),  
  
    Alcohol_Consumption: Number(v.Alcohol_Consumption),  
    Fruit_Consumption: Number(v.Fruit_Consumption),  
    Green_Vegetables_Consumption: Number(v.Green_Vegetables_Consumption),  
    FriedPotato_Consumption: Number(v.FriedPotato_Consumption),  
  };  
  
  const res = await postPredict(payload);  
  
  setResult({ ...res, input: payload });  
  nav("/results");  
};
```

0.96%

Estimated Risk Probability

Low Risk of Heart Disease

Model threshold: 0.202

```
# Output response schema
class PredictionResponse(BaseModel):
    prediction: int
    probability: float
    threshold_used: float
```

app.py

app.py

```
# Inference route
@app.post('/predict', response_model=PredictionResponse)
def predict(data: PatientData):
    try:
        # Covert input to dict
        input_dict = data.model_dump()
```

.....

```
return PredictionResponse(
    prediction=prediction,
    probability=round(probability, 4),
    threshold_used=threshold
)
```

```
function normalizePredictResponse(raw) {
  let probability = raw?.probability;
  let prediction = raw?.prediction;
  let threshold_used = raw?.threshold_used;

  if (probability == null && raw?.risk_probability != null) {
    probability = raw.risk_probability;
  }

  if (prediction == null && raw?.risk_level) {
    prediction = raw.risk_level === "High" ? 1 : 0;
  }

  if (threshold_used == null) {
    threshold_used = raw?.threshold ?? 0.5;
  }

  return {
    probability: Number(probability ?? 0),
    prediction: Number(prediction ?? 0),
    threshold_used: Number(threshold_used ?? 0.5),
  };
}
```

```
export async function postPredict(payload) {
  const { data } = await api.post("/predict", payload);
  return normalizePredictResponse(data);
}
```

client.js

Results.jsx

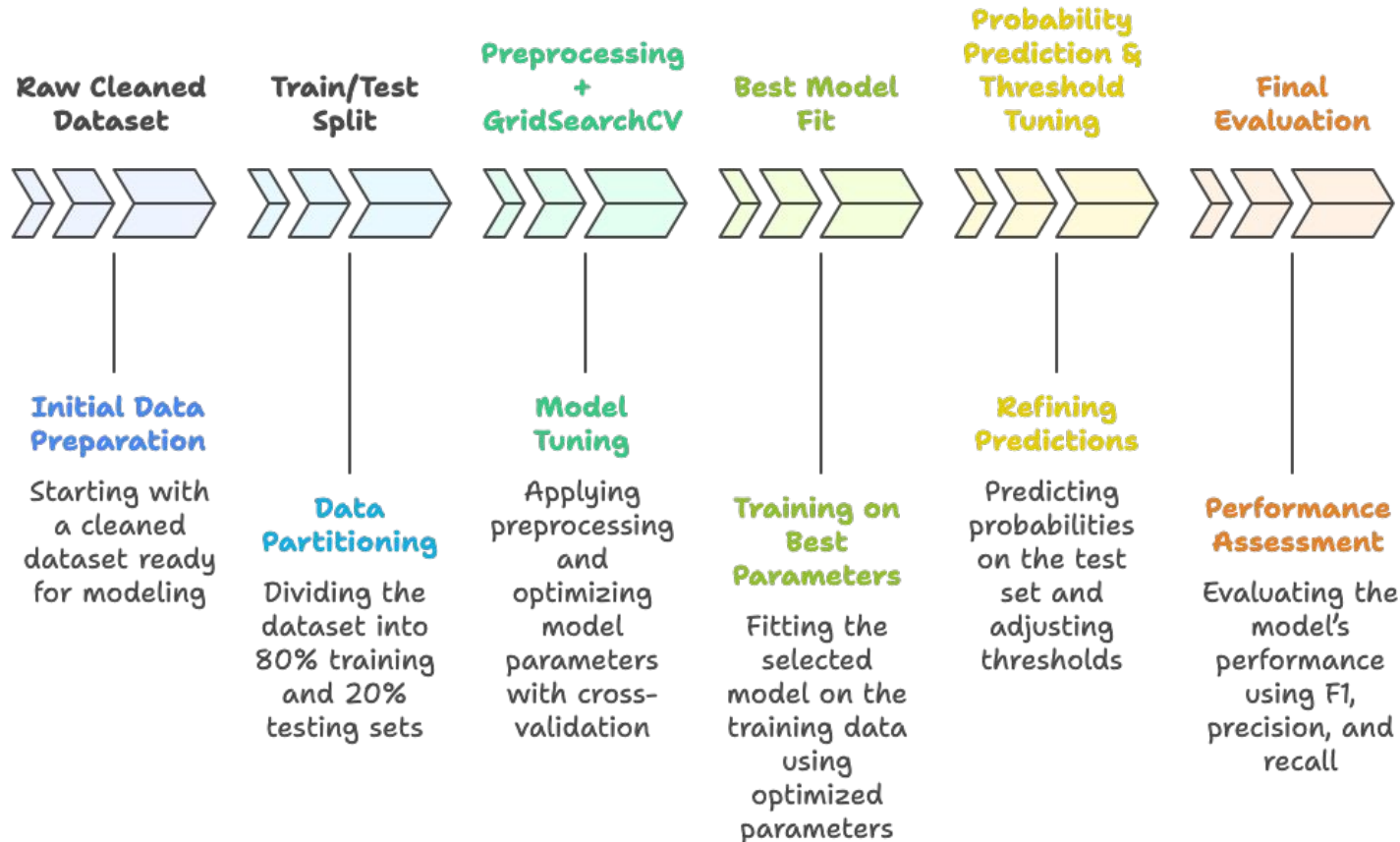
```
const prob = Number(result?.probability ?? 0); // 0~1
const pred = Number(result?.prediction ?? 0); // 0 or 1
const thr = Number(result?.threshold_used ?? 0.5); // e.g. 0.202
const percent = `${(prob * 100).toFixed(2)}%`;
const status = pred === 1 ? "High Risk of Heart Disease" : "Low Risk of Heart Disease";
```


app.py

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "https://our-frontend-domain.com"
    ], # Replace with frontend domain in prod
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Thank you :)

Machine Learning Model Development Workflow



Design Patterns used in ML Pipeline

Factory Pattern

- Used in `model_trainer.py` to dynamically select models

Pipeline Pattern

- Used in `data_transformation.py` with sklearn Pipelines

Strategy Pattern

- Evaluation strategy (`roc_auc`, `accuracy`) can be swapped easily

Builder Pattern

- Step-by-step construction of transformation pipelines

Challenges Faced

Handling ordinal vs nominal categorical data

Feature skewness and data imbalance

Balancing model performance with inference time